FPT Education

FPT POLYTECHNIC

cdio™

Conceive Design Implement Operate

Bài 6: Xử Lý Dữ Liệu Ảnh

www.poly.edu.vn

AGENDA

- Dữ liệu ảnh
- Luồng xử lý ảnh
- Kỹ thuật tiền xử lý
- Thực hành

1. Dữ Liệu Ảnh

1.1 Đặc điểm

- Ảnh số là cách máy tính nhìn thế giới: nó chỉ thấy các giá trị số biểu diễn độ sáng và màu tại từng điểm ảnh (pixel).

- Mỗi ảnh là một ma trận số học, nhưng mang nội dung trực quan – con người nhìn thấy hình ảnh, còn máy chỉ thấy dữ liệu.

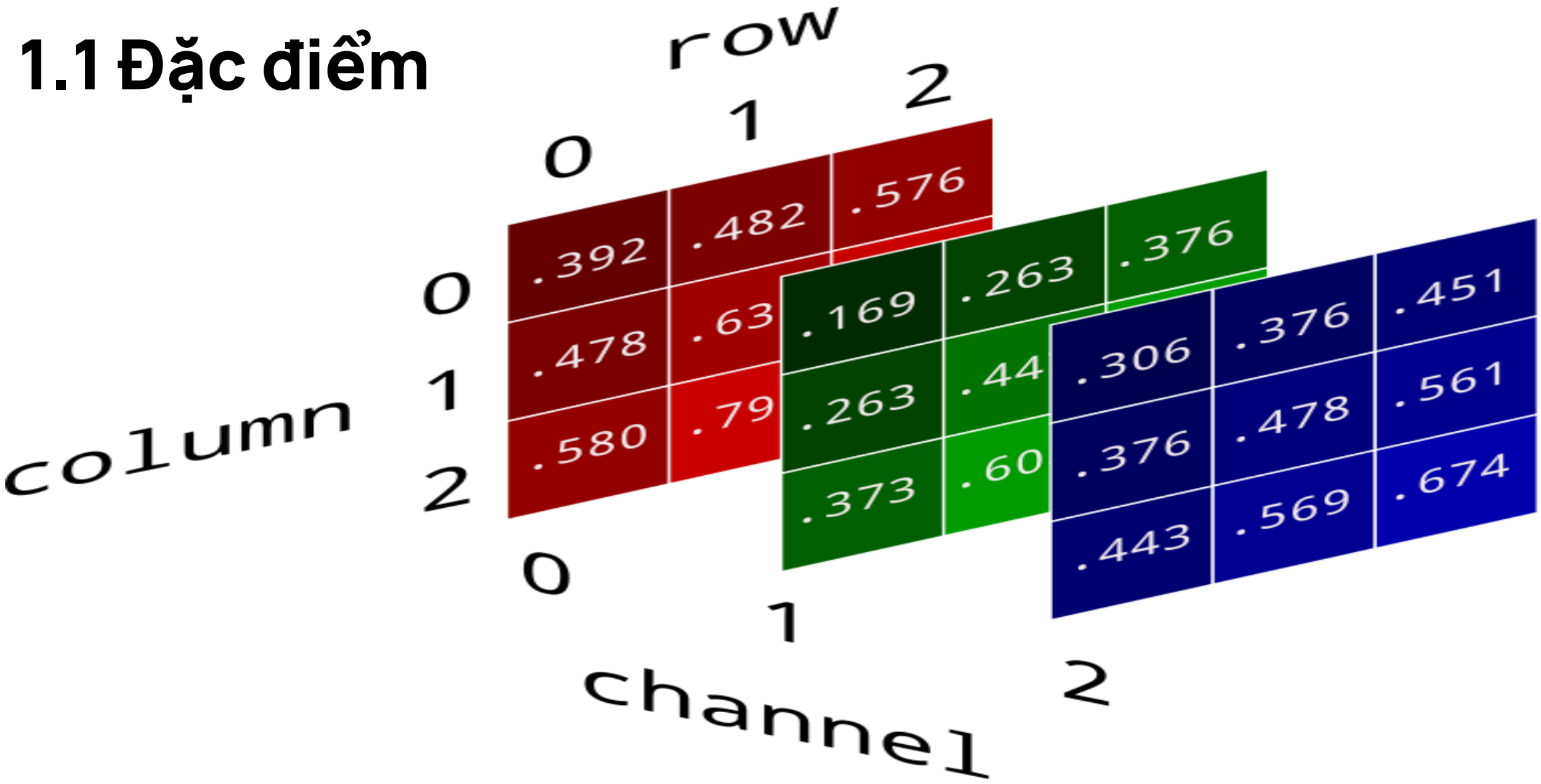
1.1 Đặc điểm

- Khi phóng to một bức ảnh-› thấy các ô vuông nhỏ -› đó là pixel.

- Một pixel sáng có giá trị gần 255 (trắng), pixel tối có giá trị gần 0 (đen).

- Ảnh màu thì mỗi pixel gồm ba giá trị (đỏ, lục, lam) → ví dụ pixel màu da có thể là (210, 160, 140).

1.1 Đặc điểm


row
0
1
2
column
0
1
2
.392
.482
.576
.478
.63
.580
.79
.169
.263
.376
.263
.44
.373
.60
.306
.376
.451
.376
.478
.561
.443
.569
.674
0
1
2
channel

1.2 Phân loại

Loại ảnh	Đặc điểm	Ứng dụng
Ảnh mức xám	Mỗi pixel 1 giá trị độ sáng	Nhận dạng chữ, xử lý y tế
Ảnh màu RGB	3 kênh (Red, Green, Blue)	Ảnh thông thường
Ảnh đa phổ (multispectral)	4–13 kênh, thu ở nhiều dải bước sóng	Ảnh vệ tinh, nông nghiệp chính xác
Ảnh siêu phổ (hyperspectral)	Hàng trăm kênh	Phân tích vật liệu, y học
Ảnh nhị phân	Pixel chỉ có 0 hoặc 1	Phát hiện biên, phân vùng đối tượng

1.3 Cấu trúc

Mỗi bức ảnh số được biểu diễn bằng ma trận (2D hoặc 3D), trong đó mỗi phần tử là một pixel. Mỗi pixel lưu giá trị biểu diễn độ sáng hoặc màu sắc.

- Với ảnh mức xám (Grayscale):

- Là ma trận 2 chiều
- Với 0 = đen hoàn toàn, 255 = trắng hoàn toàn.

1.3 Cấu trúc

Ví dụ

Ảnh mức xám 4×4:

```
[[  0,  64, 128, 255],
 [ 32, 100, 160, 220],
 [ 10,  80, 140, 200],
 [  0,  50, 110, 180]]
```

->Mỗi số là cường độ sáng của 1 pixel.

->Ô có giá trị càng lớn sẽ càng sáng.

1.3 Cấu trúc

- Với ảnh màu RGB

- Là mảng 3 chiều
- Ảnh màu thường dùng 3 kênh (Red – Green – Blue).
- Mỗi điểm ảnh có 3 giá trị (R, G, B) từ 0–255.
- Mỗi kênh là một ma trận độ sáng riêng, chồng lên nhau tạo thành ảnh màu.

1.3 Cấu trúc

- Với ảnh màu RGB

- Chiều 1 → chiều cao (height)
  Chiều 2 → chiều rộng (width)
  Chiều 3 → chiều màu (số kênh: 3)

- Kênh 1 -> Đỏ (Red)
  Kênh 2 -> Xanh lục (Green)
  Kênh 3 -> Xanh dương (Blue)

1.3 Cấu trúc

Thuật ngữ	Nghĩa đơn giản	Vai trò
Chiều (dimension)	Là hướng mở rộng của dữ liệu – cho biết mảng có bao nhiêu trục (hàng, cột, độ sâu...)	Cấu trúc tổng thể của dữ liệu
Kênh (channel)	Là tập hợp các giá trị tại cùng vị trí không gian nhưng ở các "loại thông tin" khác nhau (như R, G, B)	Biểu diễn loại thông tin của mỗi pixel

1.3 Cấu trúc

Ví dụ

Kênh R

```
[[255, 200, 150],
 [255, 100,  50],
 [200, 150, 100]]
```

Kênh G

```
[[  0,  50, 100],
 [150, 200, 255],
 [ 50, 100, 150]]
```

Kênh B

```
[[  0,   0,  50],
 [100, 150, 200],
 [255, 200, 150]]
```

1.3 Cấu trúc

Ví dụ

Ảnh màu RGB (3x3x3):

[
[[(255, 0, 0), (200, 50, 0), (150, 100, 50)],
[(255,150,100), (100,200,150), (50,255,200)],
[(200,50,255), (150,100,200), (100,150,150)]]
]

Kênh R
[[255, 200, 150],
[255, 100, 50],
[200, 150, 100]]

2. Luồng Xử Lý Ảnh

2.1 Mục tiêu

Biến ảnh thô (raw image) — thường bị nhiễu, không đồng nhất, hoặc sai định dạng — thành ảnh chuẩn hóa và đầy đủ thông tin để:

- Dễ phân tích (thống kê, hiển thị),
- Làm đầu vào cho các mô hình học máy / học sâu,
- Giảm sai lệch do thiết bị chụp hoặc môi trường.

2.2 Các giai đoạn chính

Giai đoạn	Mục tiêu	Ví dụ minh họa
Thu thập dữ liệu ảnh (Image Acquisition)	Lấy ảnh từ cảm biến, camera, file, hoặc cơ sở dữ liệu	Ảnh chụp khuôn mặt từ camera, ảnh vệ tinh từ NASA
Tiền xử lý (Pre-processing)	Làm sạch ảnh, giảm nhiễu, cân bằng sáng, chuẩn hóa kích thước và định dạng	Lọc nhiễu, resize về 224×224, chuyển RGB → Gray
Tăng cường dữ liệu (Data Augmentation)	Tạo thêm biến thể hợp lý của ảnh để tăng độ đa dạng và khả năng tổng quát	Lật ngang, xoay 10°, thay đổi độ sáng, thêm nhiễu nhẹ
Trích chọn đặc trưng (Feature Extraction)	Biến ảnh thành đặc trưng có ý nghĩa để mô hình dễ học	Trích đặc trưng biên, góc, màu sắc, hoặc dùng CNN để lấy embedding
Phân tích / Mô hình hóa (Analysis / Modeling)	Sử dụng mô hình học máy / học sâu để phân loại, phát hiện, phân vùng ảnh	Sử dụng CNN, ViT, hoặc K-Means phân cụm vùng ảnh
Đánh giá và trực quan hóa (Evaluation & Visualization)	Kiểm tra chất lượng, hiển thị kết quả	Hiển thị ảnh gốc và ảnh đã xử lý, vẽ confusion matrix

2.2 Các giai đoạn chính

Bước	Kỹ thuật thường dùng	Ý nghĩa
Chuẩn hóa kích thước	Resize, crop, padding	Đồng nhất đầu vào cho mô hình
Giảm nhiễu	Lọc trung bình, Gaussian blur, median filter	Làm mịn ảnh, loại bỏ điểm nhiễu
Cân bằng sáng	Histogram Equalization, Gamma Correction	Giảm chênh lệch ánh sáng giữa các vùng
Chuyển đổi không gian màu	RGB ↔ Gray ↔ HSV ↔ LAB	Phù hợp với bài toán (nhận dạng biên, phát hiện màu,...)
Chuẩn hóa giá trị pixel	Chia 255 hoặc chuẩn hóa Z-score	Giúp mô hình học ổn định hơn
Cắt vùng ROI (Region of Interest)	Lọc lấy phần ảnh quan trọng	Tập trung vào vùng có thông tin (ví dụ: khuôn mặt, biển số xe)

2.3 Ví dụ - Nhận dạng khuôn mặt

Bước tiền xử lý	Ý nghĩa
Chuyển sang xám	Loại bỏ màu không cần thiết, tập trung vào hình dạng khuôn mặt.
Cân bằng sáng	Khuôn mặt chụp trong ánh sáng khác nhau → tăng độ tương phản, biên rõ hơn.
Resize 128×128	Chuẩn hóa kích thước ảnh → mô hình CNN học đồng nhất, batch dễ dàng.
Tăng cường dữ liệu (xoay/lật)	Khuôn mặt có thể nghiêng, lật → tăng khả năng tổng quát của mô hình.

2.3 Ví dụ - Nhận diện biển số xe

Bước tiền xử lý	Ý nghĩa
Chuẩn hóa kích thước	Ảnh nhiều bệnh viện, kích thước khác nhau → chuẩn hóa để mô hình học đồng nhất.
Lọc nhiễu	Giảm nhiễu → mô hình nhận diện tổn thương tốt hơn.
Chuẩn hóa pixel	Pixel đa dạng (0–4095) → gradient descent ổn định, học hiệu quả hơn.
Tăng cường dữ liệu	Xoay, lật → tránh overfitting, mô hình tổng quát.

2.3 Ví dụ - Phân loại ảnh y tế

Bước tiền xử lý	Ý nghĩa
Chuẩn hóa kích thước	Ảnh nhiều bệnh viện, kích thước khác nhau → chuẩn hóa để mô hình học đồng nhất.
Lọc nhiễu	Giảm nhiễu → mô hình nhận diện tổn thương tốt hơn.
Chuẩn hóa pixel	Pixel đa dạng (0–4095) → gradient descent ổn định, học hiệu quả hơn.
Tăng cường dữ liệu	Xoay, lật → tránh overfitting, mô hình tổng quát.

3. Kỹ Thuật Xử Lý Ảnh

3.1 Chuyển ảnh xám

Mục đích: Giảm số kênh màu, giữ hình dạng và biên.

Phương pháp:

- Công thức: *gray = 0.299*R + 0.587*G + 0.114*B*
- Duyệt từng pixel, tính giá trị xám và tạo ma trận mới.

3.1 Chuyển ảnh xám

```
# Thuần Python
gray_image_thuan = np.zeros((rgb_image.shape[0], rgb_image.shape[1]), dtype=np.uint8)
for i in range(rgb_image.shape[0]):
    for j in range(rgb_image.shape[1]):
        R,G,B = rgb_image[i,j]
        gray_image_thuan[i,j] = int(0.299*R + 0.587*G + 0.114*B)

# OpenCV
gray_image_cv = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2GRAY)
```

3.2 Cân bằng sáng

Mục đích: Ảnh tối hoặc quá sáng → tăng độ tương phản, chi tiết nổi bật.

Phương pháp:

- Tính histogram (số pixel mỗi giá trị 0–255)
- Tính CDF (cumulative distribution function)
- Map lại giá trị pixel → cân bằng sáng

3.2 Cân bằng sáng

```
# Thuần Python
hist = [0]*256
for row in gray_image_thuan:
    for p in row:
        hist[p] +=1

cdf = []
cumsum = 0
total = gray_image_thuan.size
for h in hist:
    cumsum += h
    cdf.append(cumsum/total)

eq_image_thuan = np.zeros_like(gray_image_thuan)
for i in range(gray_image_thuan.shape[0]):
    for j in range(gray_image_thuan.shape[1]):
        eq_image_thuan[i,j] = int(cdf[gray_image_thuan[i,j]]*255)

# OpenCV
eq_image_cv = cv2.equalizeHist(gray_image_cv)
```

3.3 Chỉnh kích thước

Mục đích: Chuẩn hóa kích thước → input đồng nhất, batch dễ xử lý.

Phương pháp:

- Thuật toán Nearest Neighbor
- Với mỗi pixel mới, tìm pixel gần nhất từ ảnh cũ.

3.3 Chỉnh kích thước

```python
# Thuần Python (Nearest neighbor)
def resize(image, new_h, new_w):
    old_h, old_w = image.shape[:2]
    resized = np.zeros((new_h,new_w), dtype=image.dtype)
    for i in range(new_h):
        for j in range(new_w):
            old_i = int(i*old_h/new_h)
            old_j = int(j*old_w/new_w)
            resized[i,j] = image[old_i, old_j]
    return resized

resized_thuan = resize(eq_image_thuan, 128,128)
# OpenCV
resized_cv = cv2.resize(eq_image_cv, (128,128))
```

3.4 Xoay/Lật ảnh

Mục đích: Tăng số lượng và đa dạng dữ liệu → tránh overfitting.

Phương pháp:

- Xoay -> Biến đổi tọa độ pixel quanh tâm ảnh sử dụng ma trận quay.
- Lật -> Đảo thứ tự pixel theo một chiều.

3.4 Xoay/Lật ảnh

```python
# Lật ngang
def flip_horizontal(img):
    return [row[::-1] for row in img]

# Lật dọc
def flip_vertical(img):
    return img[::-1]
```

```python
# Xoay 90°
def rotate_90_clockwise(img):
    rows = len(img)
    cols = len(img[0])
    rotated = [[0]*rows for _ in range(cols)]
    for i in range(rows):
        for j in range(cols):
            rotated[j][rows-1-i] = img[i][j]
    return rotated
```

3.4 Xoay/Lật ảnh

```python
# Sử dụng NumPy

# Lật ngang (horizontal)
flipped_h = np.fliplr(image)

# Lật dọc (vertical)
flipped_v = np.flipud(image)

# Xoay 90° clockwise
rotated_90 = np.rot90(image, k=-1)
```

```python
# Sử dụng OpenCV

# Lật ngang
flipped_h_cv = cv2.flip(image, 1)

# Lật dọc
flipped_v_cv = cv2.flip(image, 0)

# Xoay một góc tùy ý, ví dụ 30 độ
h, w = image.shape[:2]
center = (w//2, h//2)
angle = 30
scale = 1.0

# Ma trận xoay
M = cv2.getRotationMatrix2D(center, angle, scale)
rotated_cv = cv2.warpAffine(image, M, (w, h))
```

3.5 Chuẩn hóa pixel

Mục đích: Pixel khác nhau → gradient khó hội tụ -> giúp mạng học ổn định.

Phương pháp: Chia 255 → pixel trong [0,1]

3.5 Chuẩn hóa pixel

```
# Thuần Python
normalized = [[pixel/255.0 for pixel in row] for row in image]

# NumPy / OpenCV
normalized = image / 255.0
```

3.6 Cắt ảnh

Mục đích: Loại bỏ phần không cần thiết, giảm nhiễu.

Phương pháp:

- Xác định tọa độ vùng cắt: (x_start, y_start) → (x_end, y_end)
- Lấy pixel con trong ma trận ảnh đó.

3.6 Cắt ảnh

```python
# Thuần Python
def crop(img, x_start, y_start, x_end, y_end):
    return [row[y_start:y_end+1] for row in img[x_start:x_end+1]]
cropped = crop(image, 1, 1, 3, 3)

# NumPy/OpenCV
cropped = image[1:4, 1:4]
```

FPT Education

FPT POLYTECHNIC

THANK YOU